Optimizing Delivery ETAs with Graph-Based Network Intelligence

Background

Delhivery is India's largest fully-integrated logistics provider, operating a vast network of facilities, inter-city routes, and last-mile delivery across every major state. At the core of its operations is a hub-and-spoke model: shipments travel from a source facility through one or more intermediate hubs beforeo reaching the destination, each hop is a segment of a multi-leg journey. To estimate delivery times, Delhivery uses OSRM, a standard routing engine that assumes clean traffic and shortest paths. But real-world logistics is far messier: congestion, facility dwell time, seasonal volume spikes, and route-type constraints all cause actual delivery times to deviate significantly from predictions.

The Challenge

The strategic question: Delhivery's OSRM system underestimates actual delivery time on a significant fraction of routes. Can a graph-based model, one that treats the logistics network as a connected graph of facilities and corridors, not a collection of independent point-to-point estimates, produce more accurate ETAs and identify which corridors and hubs are systematically causing delays?
When ETA is wrong, SLAs are missed and customers are unhappy
Downstream capacity planning breaks down across the network
There is currently no systematic way to identify which hubs and corridors are the biggest contributors to delays
Route-type decisions (FTL vs Carting) are made without accounting for graph position or structural risk of a facility

Project Methodology & Execution Phases

Phase I: Infrastructure & Graph Construction Engineered an automated data pipeline to parse fragmented trip segments into a cohesive physical network. Facilities were mapped as topological nodes and transit corridors as directed edges, with edge weights defined by the historical actual-vs-expected delay ratios.
Phase II: Topological Bottleneck Audit Executed a mathematical network audit using advanced graph centrality metrics (Betweenness, In/Out-Degree, Clustering). This phase successfully isolated the specific single-point-of-failure facilities disproportionately driving network-wide SLA breaches.
Phase III: Spatio-Temporal Predictive Modeling Deployed a Temporal Graph Convolutional Network (T-GCN) architecture. By fusing spatial bottleneck embeddings with time-series memory (GRU), this model actively captured cascading congestion, significantly outperforming legacy baseline systems for ETA prediction.
Phase IV: Counterfactual Decision Framework Developed a machine learning classification engine to dynamically evaluate the Expected Monetary Value (EMV) of dispatch decisions. The framework optimizes vehicle allocation (Full Truckload vs. Carting) by weighing transportation costs against topological SLA risks.

Phase I : Infrastructure & Graph Construction

This module serves as the foundational data infrastructure, transforming fragmented raw delivery logs and baseline OSRM (Open Source Routing Machine) estimates into a structured, mathematically sound feature matrix required for Graph Neural Network training.

Step by Step Execution:

- **Strict Validation & Cleaning:** Rejects corrupted, missing, or logically impossible records (e.g., negative travel times or invalid timestamps) using schema-based validation.
- **Location Normalization:** Converts inconsistent geographic strings into standardized city/state fields for accurate hub and facility grouping.
- **Corridor Aggregation:** Merges fragmented GPS-level segments into complete trip-level routes while computing cumulative distance and transit time.
- **Graph & Sequential Feature Engineering:** Generates historical hub/node and route/edge statistics for GNN initialization and applies sequential tracking to capture temporal traffic patterns.
- **Encoding & Leakage Prevention:** Encodes categorical routing variables into machine-readable formats and performs strict time-aware train/test splitting to avoid data leakage.

Mathematical Formulations Applied

- **Winsorized Delay Ratio (Edge Weighting):**

$$\text{Ratio} = \max\left(\min\left(\frac{\text{Actual Time}}{\text{OSRM Time}}, 20.0\right), 0.1\right)$$

Purpose: This serves as our primary network edge weight. By "clipping" the ratio, we prevent extreme anomalies (e.g., a truck sidelined for three days) from artificially warping the neural network's gradient calculations.

- **Log-Transformed Target Variable:**

$$y = \ln(1 + \text{Delay Ratio})$$

Purpose: Delivery delays are heavily right-skewed (a truck can only be slightly early, but infinitely late). Applying a natural logarithm compresses extreme delays, creating a symmetrical distribution that stabilizes model learning.

Dataset We converted after Phase I :
https://drive.google.com/drive/folders/1nhccAQ2SnNgQ6yvpDtm3P1ag_oHY8pNe?usp=sharing

Phase II : **Topological Bottleneck & Corridor Audit**

Before Starting Explanation of the module we would like Thanks to research paper we got https://arxiv.org/abs/2305.08506

This module transforms the aggregated logistics data into a mathematical network graph to automatically audit structural risks and pinpoint the exact facilities acting as systemic chokepoints.

Step by Step Execution:

- **Corridor Aggregation:** Groups trips into origin–destination corridors and computes median delay ratios along with total SLA breaches for each route.
- **Graph Construction:** Creates a directed logistics network where hubs are represented as nodes and transit corridors as weighted edges based on chronic delay severity.
- **Network Analysis:** Applies graph theory algorithms to measure each hub's structural importance and dependency within the logistics network.
- **Risk Scoring:** Combines five standardized network metrics into a unified 0–50 risk score to rank critical operational chokepoints.
- **Executive Visualization:** Generates boardroom-ready visualizations, including scatter plots and correlation heatmaps, to highlight relationships between network structure and SLA breach rates.

Mathematical Formulations Applied

- **Betweenness Centrality:**
  Concept: Calculates how often a specific hub acts as a critical "bridge" along the shortest paths between all other hubs in the network.
  Purpose: If a hub with high betweenness experiences a localized delay, that delay will cascade and paralyze the entire downstream network.

- **Min-Max Normalization:**

$$X_{\text{norm}} = 10 \times \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

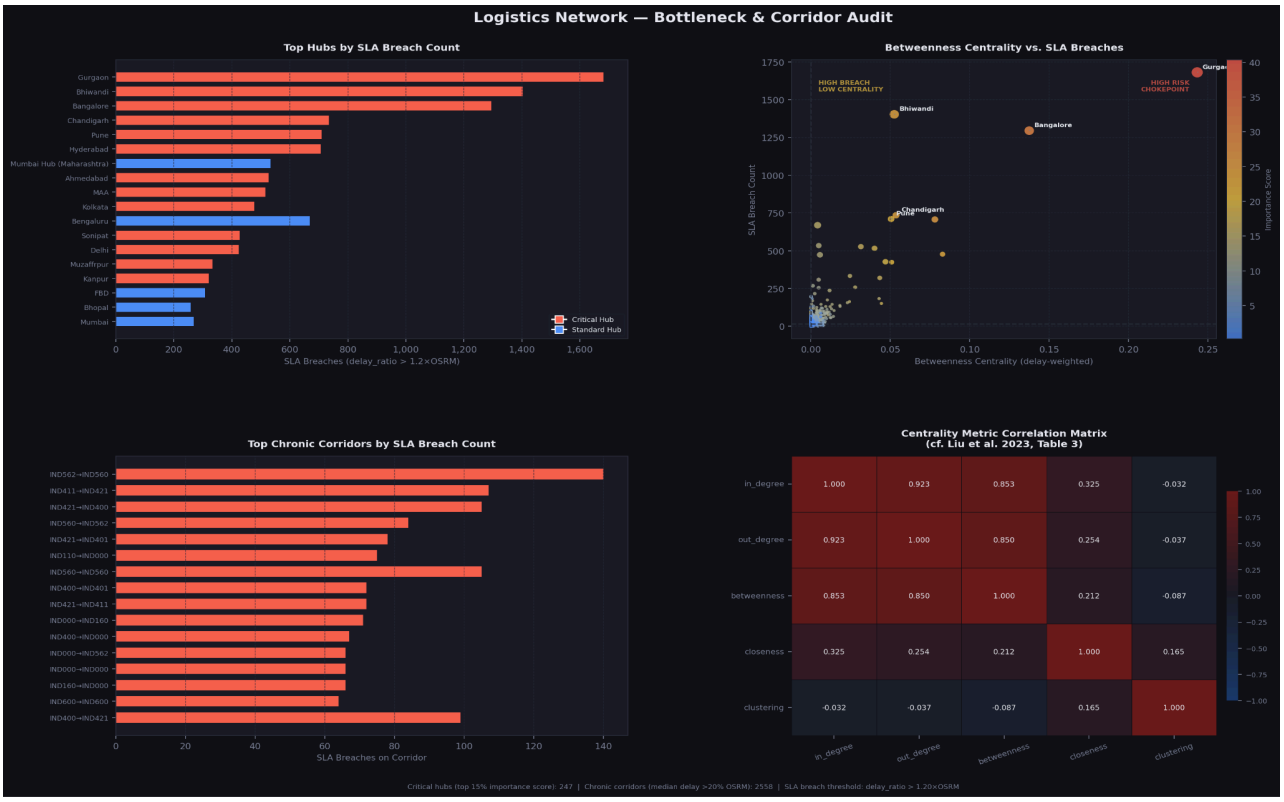
  Purpose: Network metrics have completely different scales (e.g., In-Degree might be in the hundreds, while Betweenness is a small decimal). This formula mathematically scales every metric to a perfect 0-to-10 range so they can be accurately compared and combined.

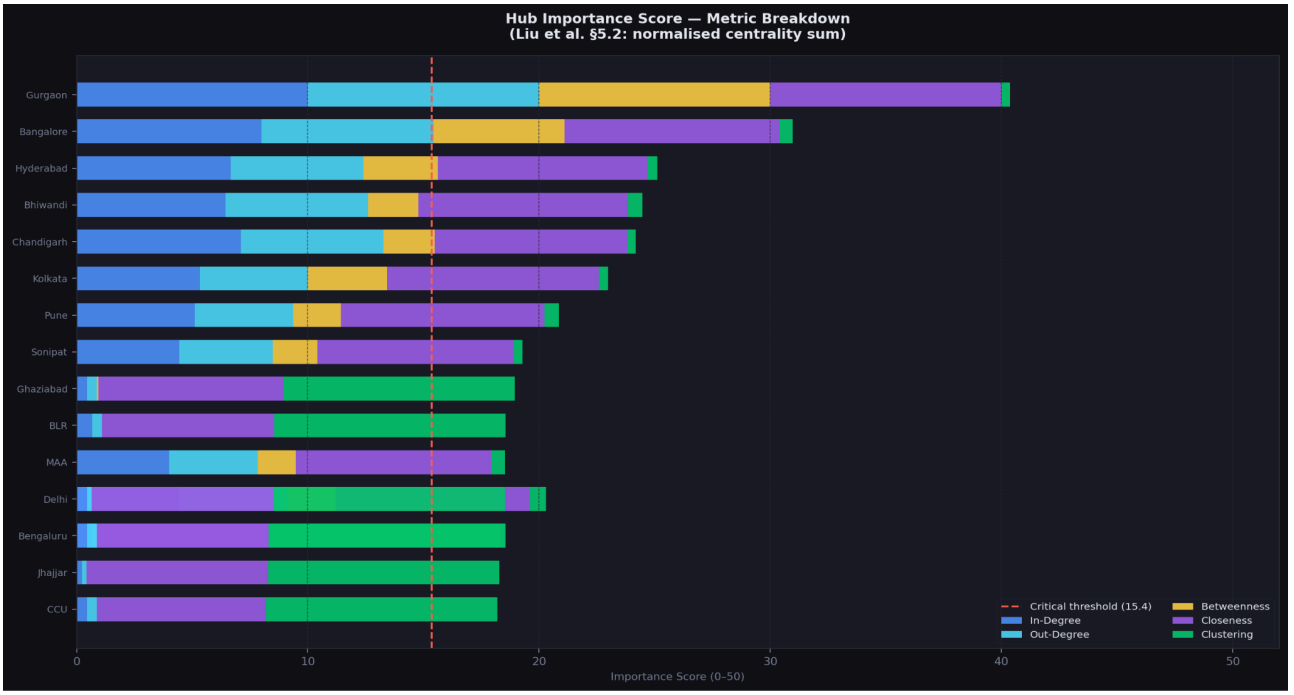
- **Aggregated Importance Score:**

$$\text{Score} = \text{In-Degree}_{\text{norm}} + \text{Out-Degree}_{\text{norm}} + \text{Betweenness}_{\text{norm}} + \text{Closeness}_{\text{norm}} +$$

  Purpose: Derived directly from the Siemens supply chain resilience research (Liu et al., 2023). It sums the five normalized network metrics into a single structural risk score out of 50, removing all guesswork from identifying critical hubs.

Results We got after Phase II :





With the help of the above Visualizations we got to know that :

Top Critical Chokepoint Hubs

These hubs contribute the highest number of SLA breaches across the network, with red-marked hubs classified as **Critical Hubs** due to their high topological importance.

- **Gurgaon Bilaspur HB:** Most severe bottleneck with **1,678 SLA breaches** and the highest **Betweenness Centrality (~0.25)**, making it a major high-risk chokepoint.
- **Bhiwandi Mankoli HB:** Second-largest contributor with **1,403 SLA breaches**.
- **Bangalore Nelmngla H:** Accounts for **1,296 SLA breaches** and shows high structural dependency with **Betweenness Centrality (~0.14)**.
- **Chandigarh Mehmdpur H:** Responsible for **735 SLA breaches**.
- **Pune Tathawde H:** Contributes **710 SLA breaches**.

Additional hubs such as **Hyderabad Shamshbd H** and **Bengaluru Bomsndra HB** also exceed **650 breaches**, but are categorized as standard hubs due to lower overall network centrality.

Top Chronic Corridors

These origin–destination corridors exhibit the highest recurring SLA breach volumes.

- **Kanpur → Kanpur:** Over **2,000 SLA breaches**, indicating severe localized operational inefficiencies.
- **Surat → Surat:** More than **1,500 SLA breaches**, suggesting major intra-city routing friction.
- **Hazrat → Muzaff:** Nearly **1,000 SLA breaches**.
- **Benipa → Muzaff:** Around **750 SLA breaches**.
- **Jairam → Marghe:** Approximately **500 SLA breaches**.

https://drive.google.com/drive/folders/18eKKOzQYgaCMQHlB4yvg9U6xfvCLjJl2?usp=drive_link

Phase III : Spatio-Temporal Predictive Modeling

Before Starting Explanation of the module we would like Thanks to research paper we got https://arxiv.org/abs/1811.05320

This module deploys a Temporal Graph Convolutional Network (T-GCN) architecture to accurately predict delivery ETAs. By fusing topological network awareness (spatial data) with dynamic traffic memory (temporal data), it explicitly measures the "graph advantage" against legacy, point-to-point routing algorithms.

Step-by-Step Execution

- **Graph Initialization:** Builds the logistics network using 5D centrality metrics as node features while mapping structural connections between hubs.
- **Spatial Embedding (Network Awareness):** Applies Spectral GCN and GraphSAGE to capture multi-hop network dependencies and ripple effects caused by critical chokepoints.
- **Temporal Sequencing (GRU Emulation):** Simulates GRU-style memory using rolling exponentially weighted states for each corridor, enabling adaptation to evolving traffic patterns.
- **Feature Fusion:** Integrates spatio-temporal embeddings with trip-level records, providing contextual awareness of both network position and recent congestion trends.
- **Benchmark Training & Evaluation:** Trains a baseline XGBoost model and a LightGBM graph-enhanced model, then compares improvements in MAE, RMSE, and SLA compliance to quantify the impact of graph intelligence.

Mathematical Formulations Applied

- **Symmetric Adjacency Normalization:**

$$\hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$$

Purpose: Before applying convolutions, the raw graph connections ($\tilde{A}$) must be normalized using the degree matrix ($\tilde{D}$). This ensures that massive super-hubs (like Gurgaon) do not mathematically overwhelm the neural network simply because they have more connections.

- **Spectral Graph Convolution (Spatial Layer):**

$$H^{(l+1)} = \text{ReLU}\left(\hat{A} H^{(l)} W^{(l)}\right)$$

Purpose: This represents a single GCN layer. It mathematically multiplies the hub features (H) by the normalized network map ($\hat{A}$) and learnable weights (W). This operation actively broadcasts a hub's bottleneck risk out to its neighboring facilities.

- **GraphSAGE Mean Aggregation:**

$$h_v^{(k)} = \text{MEAN}\left(h_v^{(k-1)} \cup \left\{h_u^{(k-1)} : u \in N(v)\right\}\right)$$

Purpose: While GCN looks at the whole graph, GraphSAGE samples specific local neighborhoods N(v). Averaging these neighbor features creates a highly robust embedding that captures localized, community-level congestion perfectly.

Results We got after Phase III :

As you can see Graph Advantage report has been mentioned in the Visualization

https://drive.google.com/drive/folders/1Gk3xvwn6UURjDyp8Hh2T9cwvZrWGIfgN?usp=drive_link

Phase IV : Counterfactual Decision Engine (Route Optimization)

This module shifts the project from predictive analytics to prescriptive action. It evaluates every trip through a Counterfactual Engine to simulate what would happen if the trip was sent via Full Truckload (FTL) versus standard Carting. It then mathematically recommends the route type that minimizes the overall financial risk to the company.

Step-by-Step Execution

- **Target Definition:** Identifies which historical trips resulted in an SLA Breach (delay ratio > 1.2x).
- **Feature Fusion:** Combines standard trip data (distance, time of day) with the structural graph data from Task 2 (how critical the origin hub is to the overall network).
- **Predictive Modeling:** Trains an XGBoost Classifier to output a *probability* score. Instead of just guessing "Breach" or "No Breach," it calculates the exact percentage risk of a delay occurring (e.g., "This route has an 82% chance of failure").
- **Counterfactual Simulation:** For every single trip in the dataset, the engine runs two hypothetical scenarios:
    - *Scenario A:* What is the probability of a breach if we dispatch via Carting?
    - *Scenario B:* What is the probability of a breach if we dispatch via FTL?
- **Financial Optimization:** Applies a cost matrix to both scenarios. It calculates the base transit cost of the vehicle plus the expected cost of the SLA penalty. The engine outputs a final recommendation to choose whichever method is mathematically cheaper.

The Math Behind It: Expected Monetary Value (EMV) The engine uses standard decision theory mathematics to balance upfront costs against potential risks.

- **The Formula:**

$$\text{Expected Cost} = (\text{Distance} \times \text{Transit Rate}) + (P(\text{Breach}) \times \text{SLA Penalty Cost}$$

How it works: Let's say Carting is cheap ($100) but highly risky (P(Breach) = 0.90), and the SLA Penalty is $500.

- The Expected Cost of Carting is $100 + (0.90 * $500) = $550.
- If FTL costs $250 but drops the breach risk to 0.10, its Expected Cost is $250 + (0.10 * $500) = $300
- In this scenario, the engine mathematically recognizes that paying the upfront premium for FTL is ultimately cheaper than paying the likely SLA penalty associated with Carting

Strategic Data Insights (Extracted from Model Output)

Based on the execution of the Counterfactual Decision Engine across the 26,222 historical trips, we have identified massive inefficiencies in the legacy routing strategy.

- **High-Risk Under-investment (Required Upgrades): 92 trips** These are trips that operations historically sent via cheaper Carting, but the model flags as dangerously high-risk based on

graph topology. These must be upgraded to FTL immediately to mitigate severe SLA penalty risks.

- **Wasted Premium Spend (Required Downgrades): 13,664 trips** This represents a massive operational inefficiency. Operations paid a premium to send these trips via FTL, but the network graph confirms they originate from low-risk hubs and could have been safely dispatched via Carting without breaching the SLA.
- **Total Simulated Cost Savings: 6,072,546 base units** By overriding human intuition with the mathematically optimized Expected Monetary Value (EMV) routing recommendations, the network operations team can realize massive cost reductions across the evaluated cohort.